✅ 1. Java Main Class

📄 src/main/java/PrescriptionService.java

```
public class PrescriptionService {

  public static boolean isValidMedicine(String medicine) {
    if (medicine == null || medicine.trim().isEmpty()) return false;

    String[] meds = {"Paracetamol", "Ibuprofen"};
    for (String m : meds) {
      if (m.equalsIgnoreCase(medicine)) return true;
    }
    return false;
  }

  public static boolean isValidDosage(int dosage) {
    return dosage > 0 && dosage <= 1000;
  }

  public static boolean validatePrescription(String med, int dose) {
    return isValidMedicine(med) && isValidDosage(dose);
  }

  // MAIN METHOD (Required for Docker/Kubernetes)
  public static void main(String[] args) {
    System.out.println("Prescription App Running...");

    boolean result = validatePrescription("Paracetamol", 500);
    System.out.println("Validation Result: " + result);

    // Keep container alive
    while (true) {
      try {
        Thread.sleep(10000);
        System.out.println("App is running...");
      } catch (Exception e) {
        e.printStackTrace();
      }
    }
  }
}
```

✅ 2. JUnit Test Class

📄 src/test/java/PrescriptionTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PrescriptionTest {

  @Test
  void testValidPrescription() {
    assertTrue(PrescriptionService.validatePrescription("Paracetamol", 500));
  }

  @Test
  void testInvalidPrescription() {
    assertFalse(PrescriptionService.validatePrescription("Unknown", -10));
  }
}
```

✅ 3. pom.xml (VERY IMPORTANT)

```
<project xmlns="http://maven.apache.org/POM/4.0.0">

  <modelVersion>4.0.0</modelVersion>


  <groupId>com.demo</groupId>

  <artifactId>prescription-app</artifactId>

  <version>1.0</version>


  <properties>

  <maven.compiler.source>17</maven.compiler.source>

  <maven.compiler.target>17</maven.compiler.target>

  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <junit.version>5.10.1</junit.version>

  </properties>


  <dependencies>

    <dependency>

      <groupId>org.junit.jupiter</groupId>

      <artifactId>junit-jupiter</artifactId>

      <version>${junit.version}</version>
```

```
        <scope>test</scope>
      </dependency>
    </dependencies>
    <build>
    <plugins>
      <!-- Make JAR executable -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.3.0</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>PrescriptionService</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

✅ 4. Dockerfile

📄 Dockerfile

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

✅ 5. Kubernetes Deployment

📄 deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prescription-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prescription
  template:
    metadata:
      labels:
        app: prescription
    spec:
      containers:
      - name: prescription-container
        image: nivetha140805/prescription-app:latest
        imagePullPolicy: Always
```

✅ 6. Kubernetes Service

📄 service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: prescription-service
spec:
  type: NodePort
  selector:
    app: prescription
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30007
```

✅ 7. Jenkins Pipeline

📄 Jenkinsfile

```
pipeline {
  agent any

  stages {
```

```
    stage('Clone Code') {
      steps {
        git 'https://github.com/nivethas2023b-collab/prescription-app.git'
      }
    }

    stage('Build & Test') {
      steps {
        bat 'mvn clean test'
      }
    }

    stage('Package') {
      steps {
        bat 'mvn package -DskipTests'
      }
    }

    stage('Build Docker Image') {
      steps {
        bat 'docker build -t nivetha140805/prescription-app:latest .'
      }
    }

    stage('Push Docker Image') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'dockerhub-creds',
          usernameVariable: 'USERNAME',
          passwordVariable: 'PASSWORD'
        )]) {
          bat 'docker login -u %USERNAME% -p %PASSWORD%'
          bat 'docker push nivetha140805/prescription-app:latest'
        }
      }
    }

    stage('Deploy to Kubernetes') {
      steps {
        bat 'kubectl apply -f deployment.yaml'
        bat 'kubectl apply -f service.yaml'
      }
    }
  }
}
```

🏁 **YOU ARE DONE**

✓ Java Program
✓ JUnit Testing
✓ Maven Build
✓ Jenkins CI/CD
✓ Docker
✓ Kubernetes (Deployment + Service)